Json Web Token

- Json Web Token
  - Qué es JWT?
  - Ventajas de uso
  - Funcionamiento
  - Partes
  - Ejemplo
  - Header
  - Payload
    - Tipos de Claims
    - Claims mas usadas
  - Signature
  - Documentación oficial

Qué es JWT?

Autenticación + Autorización + Intercambio Seguro de Información

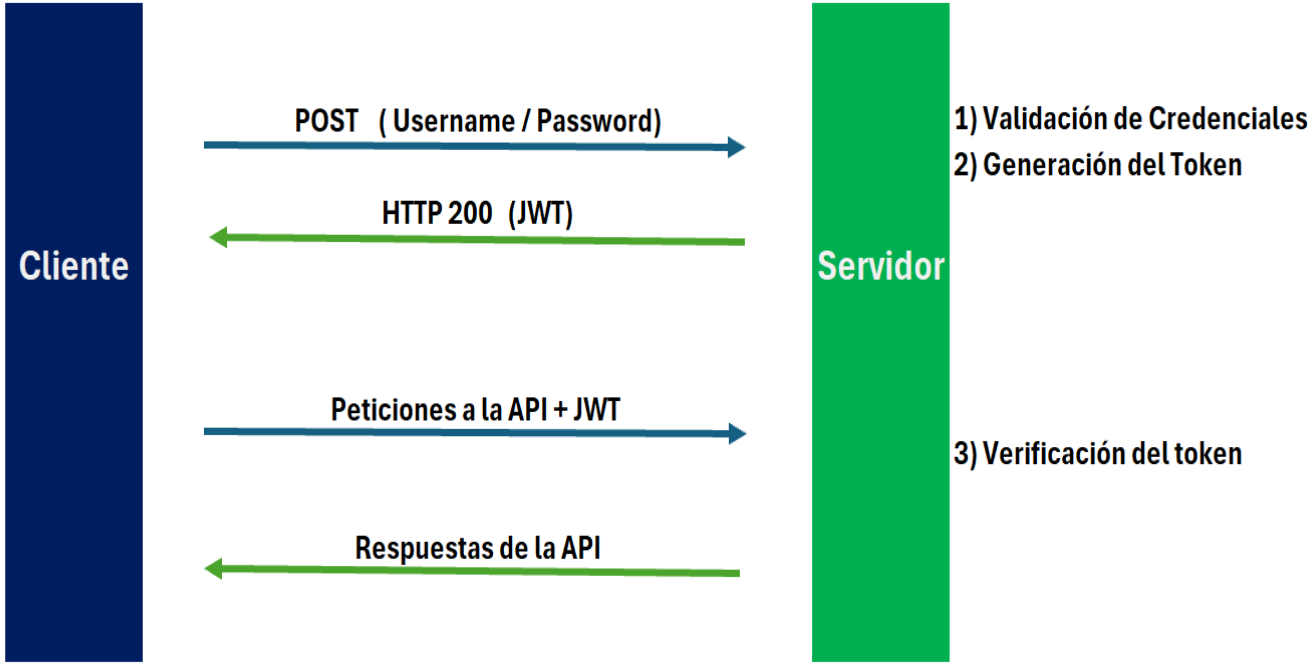
Json Web Token (JWT) es un estándar abierto basado en JSON, que nos permite crear un token.
Este token garantiza que los datos enviados entre dos partes sean válidos y seguros.
Son usados en estándares de autenticación y autorización como OAuth2 y OpenIdConnect.

Ventajas de uso

- El *costo* de transmitir el token es muy pequeño.
- Se puede usar el mismo token en diferentes dominios.
- Soporta expiración

Funcionamiento

1. Cliente envía datos de autenticación a un servidor
2. Servidor valida los datos de autenticación y genera un `JWT`
3. El `JWT` se codifica en base64 y se envía al cliente
4. Peticiones subsecuentes del cliente al servidor, deberán contener el `JWT`.
5.



> **Codificación Base64:**
> Sistema de codificación para transmitir contenido
> Usado para convertir datos binarios a un formato de solo texto conteniendo letras, números `+` y `/`.
> Posteriormente almacenarlo o transmitirlo en un medio que soporte texto

Partes

Un JWT consta de tres partes:

- **Header:** Cadena base64 creada a partir de un Json que contiene información del token.
- **Payload:** Cadena base64 creada a partir de un Json que contiene información que se desea transmitir.
- **Signature:** Cadena encriptada formada a partir de **Header** y el **Payload**.

Ejemplo

Un JWT contiene tres cadenas codificadas en base64, separadas por puntos:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4
gRG9lIiwiaXNTb2NpYWwiOnRydWV9.
4pcPyMD09olPSyXnrXCjTwXyr4BsezdI1AVTmud2fU4
```

Header

Es un Json que contiene las siguientes propiedades:

- **`alg`**: Algoritmo usado para la firma
- **`type`**: Tipo de token

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Tipos de algoritmos de cifrado mas usados:

- **Simétricos**: HMAC, SHA256 (la misma llave firma y verifica el token)
- **Asimétricos**: RSA (una llave privada lo firma y una publica lo verifica)

Payload

Es un Json que puede tener cualquier propiedad (aunque existen propiedades estándar ya definidas).
Puede incluir por ejemplo las credenciales de acceso de un usuario.
A las propiedades de este Json se les conoce como **claims**.

```
{
  "login": "userName",
  "password": "userPassword"
}
```

Tipos de Claims

- **Registered**: Los predefinidos por la especificación JWT: `issuer, expirationtime, subject`
- **Public**: Los definidos por el usuario que pueden registrarse
- **Private**: Los usados solo por el emisor y receptor en cierto caso de uso.

Claims mas usadas

En el payload podemos poner lo que queramos estos claims no son obligatorios
pero se recomienda su uso por cuestiones de interoperabilidad.

- **sub:** Usado para poner el ID de usuario (subject)
- **iss:** Usado para poner el nombre del emisor (issuer)
- **exp:** Tiempo de expiración del token

Signature

Es una firma que se genera con un algoritmo de cifrado y que verifica que el mensaje no ha sido alterado
Se toma el header(base64) y el payload(base64) se encripta con el algoritmo definido en el header
y con una clave secreta que solo esta definida en el servidor

Documentación oficial

https://jwt.io/

